HELLO!

I am Sayeed Unnisa

I am here because I love to giving presentations on **Java Full Stack**

SPRING FRAMEWORK

Spring:

- The Spring Framework is an open-source application framework for building Java-based enterprise applications.
- The core features of the Spring Framework can be used in developing any Java application
- It can be described as complete and modular framework.
- The Spring Framework can be used for all layer implementations of a real time application.
- It can also be used for the development of particular layer of application

1. Test:

Spring test module provides the supports for testing of spring components with JUnit or TestNG frameworks.

2. Core Container:

The **Core Container** is heart of Spring. It contains some base framework classes and tools. The entire Spring Framework is based on top of the **Core Container.**

Spring core container contains the following:

Spring Core

The **Core** module contains basic Spring Framework classes including Dependency Injection (DI) and Inversion of Control (IOC).

Spring Bean

Spring Bean module manages the lifecycle of beans. In the Spring Framework a Bean is any Java Class which is registered with Spring and Spring manages these bean classes.

Spring Context

Spring contexts are also called Spring IoC containers, which are responsible for instantiating, configuring, and assembling beans by reading configuration metadata from XML, Java annotations, and/or Java code in the configuration files.

3. Data Access/Integration

The Spring Data Access is a set of modules, for accessing data in various formats including Database, Messaging, and XML.

Spring JDBC

The **Spring JDBC** provides abstraction over Java JDBC API.
It provides **JdbcTemplate** to easily access data from databases

Spring ORM

Spring ORM provides support for integrating with various ORM frameworks where data is mapped to a Java Object

Spring JMS

The JMS stands for Java Messaging Service , it is a specification for Publisher and Subscriber communication in the form of messages Spring JMS provides an abstraction over various JMS implementations like ActiveMQ and RabbitMQ.

Spring Transactions

Spring Transactions Management API provides uniform way of managing transactions of data objects as well as databases.

4. Spring Web

Spring Web components are used to build web applications. Using the Spring Web module we can build complete MVC applications, interceptors, Web Services.

Spring Web & Servlet

Spring Web and Servlets provides many features for building web integrations.

Spring MVC

Spring MVC provides a mechanism for building Model View Controller based web applications.

5.Miscellaneous Modules

Spring AOP

Spring AOP is an implementation of Aspect Oriented Programming. An Aspect is any secondary task which an object needs to perform.

Spring Aspects

Spring Aspects provides a uniform way of integrating with other Aspect Oriented Programming implementations like AspecJ.

Spring Messaging

The module provides simplified and uniform way of interacting with various messaging services.

Spring Container:

Spring IoC Container is the core of Spring Framework
Spring Container takes care of

- Creating Objects
- Providing data to object
- Link object to another objects
- Destroy Objects

Spring Container needs two input from programmer.

1. Spring bean (Java Files / class)
2. Spring Config File (XML / JAVA / ANNOTATION)

Spring Bean:
Spring bean is a simple object which is created, managed and destroyed by **spring container**.

Rules for spring bean
-> Class must be public type
-> It should have a public no-arg constructor.
-> All properties in java bean must be private with public getters and setter methods.
-> class can override method from object (java.lang) class given below. toString() , hashCode() , equals()

Inversion of Control (IoC)

- If controlling of an application classes is transferred (inverted) to a third party then it is said to be inversion of control
- It is a design principle that allows classes to be loosely coupled and, therefore, easier to test and maintain.
- Inversion of control means letting a framework take control of the execution flow of your program to do things like create instances of your classes and inject the required dependencies.
- in spring framework the controlling of spring beans is inverted to the spring container so spring framework follows IOC design pattern

DI (Dependency Injection):

It is a concept is used by spring container to create objects and providing data to variables

Dependency:

It is a variable defined in a class (Spring Bean)

It is divided into three types

1. Primitive Type Dependency.
2. Collection Type Dependency
3. Reference Type Dependency

Dependency Injection(DI) :

Injection means "Providing data(values) to variable (dependency)"

It is 3 types :

1. Setter Injection
2. Constructor Injection
3. LookUp method Injection

1. Setter Injection:

- **Setter** injection is a **dependency injection** in which the spring framework injects the dependency object using the **setter** method.
- It uses default constructor and set method

Ex:

```
Class A
{
int sid ;
}
A a = new A();
a.setSid(25);
```

2. Constructor Injection(CI):

Container provides data while creating object using "Parameterised Constructor" It is called as (CI)

Ex:

```
class A{int sid;} A a1 = new A(55);
```

Spring configuration file:

- In order to tell the spring container about the application classes and their dependencies the spring configuration file is used
- The spring framework has provided simple tags to configure spring beans in xml file

XML Configuration file

<bean class = "_____" name = "_____">
<property name = "_____">
<value> _____ </value>
<property>
</bean>

1. <bean>:: Indicate object , which will be created in Spring container.
2. <property> :: It will call set method of given variable to provide data.
3. <value> :: It indicates data to variable.

Simple Spring Core First Application.

1. Spring Bean

=> Simple Java class that follows rules given by Container

2. Spring Configuration File(XML)

=> It gives object details (name, data, links with other objects ..etc)

3. Test class

=> Just find, container created or not? Object is created or not?

Spring Core XML Tags

<bean> - Object creation

<property> -- calling set method

<constructor-arg> -- call parameterized constructor

<value> ---- To provide data to Primitive Type.

<list> <set> <map> <props> -- To provide data to collection type

<ref/> --- To link two objects

1. To create object
<bean id="objName" class="FullQualifiedClassName"> </bean>

2. To call set method
<property name="variableName"> </property>

3. To provide data to Primitive Variable.
<value> SOME DATA </value>

Types of containers

Spring Containers are two Types.

1. BeanFactory(Supports only XML)
2. ApplicationContext(Supports all XML/Java/Annotation Config)

- ApplicationContext it is a interface.
- We are using one of its Impl class called ClasPathXmlApplicationContext(C).

It mean,

ClassPath = src/main/resources folder

Xml = XML Config File

ApplicationContext = Spring Container

sample Test class code

```
//creating container + objects
ApplicationContext ac = new
ClasPathXmlApplicationContext("config.xml");
//read object from container
Object ob = ac.getBean("objName");
//print object data
sysout(ob);
```

Collections Configuration

Spring container supports working with collection Configuration which are from java.util package.

These are 4.

Collection Name	Collection Type	XML Tag Name
List	Interface	<list>
Set	Interface	<set>
Map	Interface	<map> and <entry>
Properties	Class	<props> and <prop>

When a Programmer uses interface in coding, then Container chooses a default implementation for collections.

List -> ArrayList

Set -> LinkedHashSet

Map -> LinkedHashMap

Properties --> --NA—

Map is a collection of Entries. 1 Entry = 1 Key + 1 Value
-> Entry we can define in tags format (or) attributes format.

Entry key val as tags

```
<entry>
  <key>
    <value>___</value>
  </key>
  <value>___</value>
</entry>
```

Entry - key val as attributes

```
<entry key="__" value="__"/>
```

Spring Annotation Configuration:-

=> This is easy to configure

=> Faster in execution (compared to XML)

1. Stereotype Annotations:-

- These annotations informs Spring Container to Create Object.
- These annotations must be applied at class level

=> They are 5 annotations given by Spring F/w.

=> These annotations are added in Spring version 2.5

@Component : Creates object inside container.
@Repository : Creates object + Database Operations
@Service : Creates object + Transaction/Logics/calculations..etc
@Controller : Creates object + HTTP Protocol (Web Apps)
@RestController : Creates object + HTTP Protocol (Webservices)

2. Data annotations/ Basic Annotations

@Value Annotation is used to provide data (Field Inject to variables)

It has 3 usages:

- Hardcoding (Direct value to a variable)
- Reading Data from Properties/YAML File
- SpEL (Spring Expression Language)

@Autowired:

@**Autowired** annotation is used for automatic dependency injection.

@Qualifier :

- This annotation is used along with with @Autowired to avoid confusion when we have two or more beans configured for the same type.

- If we have more than one bean of the same type and want to wire only one of them then use the **@Qualifier** annotation along with **@Autowired** to specify which exact bean will be wired.

@Qualifier tells Spring which type of Animal should be injected as a dependency to the **Farm class.**
Since both Cow and Dog implement Animal, Spring must know which implementation to inject or "autowire".
Ex:

```
public class Farm {

    @Autowired
    @Qualifier(“Dog")
    Animal animal;

    public Farm(){
    }
}
```

basePackage :

It is a packageName.

=> from this package name, Spring container will search for classes from current package and all its subpackages.

=> @ComponentScan(basePackages="com.app.test")

[or]

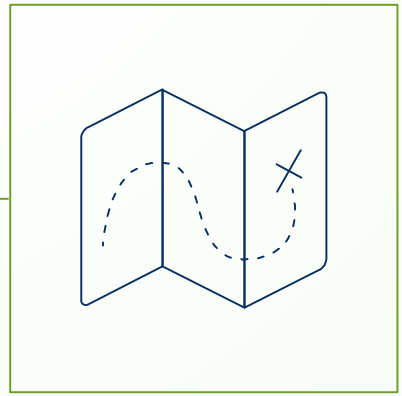
=> @ComponentScan("com.app.test")

@ComponentScan = Find for classes having stereotype annotation from given package or subpackage into Spring container.

=> Define one public class with any name
=> Apply @ComponentScan over such class
=> pass that class as input to ApplicationContext

Spring Annotation Configuration:-

1. @Component : Create Object to given class
2. @Value : To provide data (Field Injection) to variable
3. @ComponentScan : To find classes using basePackage.



THANKS!

Any questions?

You can find me at
sayeedunnisa.js@gmail.com